

DATADOG WORKFLOW AUTOMATION

Guia prático do aluno para observabilidade, alertas e automação low-code

OBJETIVO DO LABORATÓRIO

Você vai executar a aplicação CopaFigurinhas localmente, enviar eventos do navegador ao Datadog, localizar logs, transformar sinais em ALERT e acionar workflows seguros. Ao final, você deve conseguir explicar a cadeia completa: interação no /lab → log → monitor → alerta → workflow → decisão → ação.

Componente	Papel no laboratório
CopaFigurinhas /lab	Gera eventos controlados no navegador.
Datadog Browser Logs	Recebe health_check, lab_error e lab_slow.
Log Explorer	Permite pesquisar e comparar os eventos.
Log Monitor	Transforma uma consulta em condição de alerta.
Workflow Automation	Executa uma decisão e uma resposta segura quando o monitor dispara.

IMPORTANTE

Todo o laboratório deste guia é executado localmente em `http://localhost:5173`. Você não precisa de Docker nem de deploy para realizar os cenários.

PROMPT PARA ADAPTAR UM PROJETO AO LABORATÓRIO SRE COM DATADOG

Você está trabalhando em um **projeto existente**.

Seu objetivo é adicionar uma página `/lab` para gerar eventos controlados de observabilidade no Datadog, preservando integralmente a aplicação original.

REGRAS IMPORTANTES

- * Não recrie a aplicação.
- * Não altere regras de negócio.
- * Não faça refatorações desnecessárias.
- * Não altere a interface principal.
- * Não adicione dependências sem necessidade.
- * Não instale um novo framework de roteamento se o projeto já possuir uma solução adequada.
- * Não exponha Datadog API Key ou Application Key no frontend.
- * Não execute `git commit`, `git push` ou deploy.
- * Faça a menor alteração possível.

Antes de modificar qualquer arquivo, **inspecione a arquitetura real do projeto**. Não suponha que ele utiliza React, Vite, Express ou qualquer outra tecnologia.

1. DIAGNÓSTICO

Antes de alterar o código:

1. identifique frontend, backend e framework utilizados;
2. localize o arquivo de entrada da aplicação;
3. identifique como as rotas funcionam;
4. leia os `package.json` e scripts disponíveis;
5. identifique como a aplicação é executada localmente;
6. verifique como as variáveis de ambiente são carregadas;
7. procure por integração existente com `@datadog/browser-logs`;
8. execute:

```
```bash
git status
git diff
```
```

Explique brevemente a arquitetura encontrada antes de iniciar as alterações.

2. PRESERVAR A APLICAÇÃO EXISTENTE

A rota principal deve continuar funcionando exatamente como antes:

```
```text
/
```
```

Não altere:

- * regras de negócio;
- * chamadas ao backend;
- * filtros;
- * formulários;
- * relatórios;
- * autenticação;
- * APIs existentes;
- * comportamento visual da aplicação.

Se frontend e backend forem executados separadamente, descubra os comandos reais nos `package.json`.

3. CONFIGURAR O DATADOG

Verifique primeiro se já existe integração com:

```
```text
@datadog/browser-logs
```
```

Se existir, ****reutilize a configuração****.

Se não existir, implemente somente o necessário para o Datadog Browser Logs SDK.

O frontend deve utilizar apenas um **Client Token**.

Se o projeto utilizar Vite, utilize:

```
``env
VITE_DATADOG_CLIENT_TOKEN=<CLIENT_TOKEN>
``
```

e leia por:

```
``javascript
import.meta.env.VITE_DATADOG_CLIENT_TOKEN
``
```

Se utilizar outro framework, use a convenção equivalente de variável pública do frontend.

A configuração deve possuir, no mínimo:

```
``javascript
datadogLogs.init({
  clientToken: <CLIENT_TOKEN_DO_AMBIENTE>,
  site: <SITE_DATADOG>,
  service: <NOME_DO_SERVICO>,
  env: 'lab',
  forwardErrorsToLogs: true,
  sessionSampleRate: 100
});
``
```

Não coloque o token diretamente no código.

Determine um `service` coerente com o projeto e informe esse valor ao final.

4. CRIAR A PÁGINA /lab

Adicione uma página acessível em:

```
``text
/lab
``
```

Use o mecanismo de rotas já existente.

Se o projeto não possuir roteamento, implemente a solução mínima compatível com a arquitetura encontrada.

A rota `/lab` não deve interferir na aplicação original.

A página deve conter:

Título

```
``text
SRE Lab
``
```

****Descrição****

```
```text
```

Laboratório controlado para demonstrações de observabilidade e SRE.

```
```
```

****Botões****

- * Gerar evento saudável
- * Simular erro
- * Simular latência

Exiba também:

```
```text
```

Último evento: <evento>

Status: Enviado para observabilidade

```
```
```

```
---
```

5. EVENTOS DO LABORATÓRIO

Utilize a instância configurada do `datadogLogs`.

Evento saudável

Ao clicar em ****Gerar evento saudável****:

```
```javascript
```

```
datadogLogs.logger.info('health_check', {
 event_type: 'health_check',
 route: '/lab'
});
```

```
```
```

Erro controlado

Ao clicar em ****Simular erro****:

```
```javascript
```

```
datadogLogs.logger.error('lab_error', {
 event_type: 'lab_error',
 status_code: 500,
 error_type: 'SimulatedError',
 route: '/lab'
});
```

```
```
```

Esse evento não deve provocar falha real da aplicação.

Latência controlada

Ao clicar em ****Simular latência****, aguarde aproximadamente `2500 ms` e depois envie:

```
```javascript
```

```
datadogLogs.logger.warn('lab_slow', {
 event_type: 'lab_slow',
 status_code: 200,
 duration_ms: 2500,
 route: '/lab'
});

```

O objetivo é representar uma requisição que respondeu com sucesso, mas apresentou degradação de desempenho.

---

## ## 6. VALIDAÇÃO

Depois das alterações:

1. confirme que `/` continua funcionando;
2. confirme que `/lab` funciona;
3. teste os três botões;
4. execute os scripts existentes de:

```
* build;
* lint;
* testes.
```

Não invente comandos. Leia os `package.json`.

Não considere concluído se o build estiver falhando.

Se houver backend, preserve suas rotas e chamadas existentes.

---

## ## 7. VALIDAÇÃO NO DATADOG

Ao final, informe ao aluno qual `service` foi utilizado.

Os eventos devem ser pesquisáveis por:

```
```text
service:<SERVICO>
```
```

```
```text
service:<SERVICO> @event_type:health_check
```
```

```
```text
service:<SERVICO> @event_type:lab_error
```
```

```
```text
service:<SERVICO> @event_type:lab_slow
```
```

Os eventos devem conter:

**\*\*health\_check\*\***

```
```text
event_type = health_check
route = /lab
```
```

**\*\*lab\_error\*\***

```
```text
event_type = lab_error
status_code = 500
error_type = SimulatedError
route = /lab
```
```

**\*\*lab\_slow\*\***

```
```text
event_type = lab_slow
status_code = 200
duration_ms = 2500
route = /lab
```
```

---

## ## 8. ROTA /lab EM PRODUÇÃO

Verifique como o frontend é servido.

Se o projeto utilizar SPA, confirme se uma requisição direta para `/lab` precisaria de fallback para `index.html`.

Se for necessário configurar esse fallback, preserve todas as rotas de API.

Nunca redirecione:

```
```text
/api/*
```
```

para `index.html`.

Não faça deploy.

---

## ## ORDEM DE EXECUÇÃO

Siga esta ordem:

1. analisar a arquitetura;
2. executar `git status` e `git diff`;
3. validar a aplicação existente;
4. verificar ou configurar Datadog;
5. implementar `/lab`;

6. implementar os três eventos;
7. validar novamente `/`;
8. validar `/lab`;
9. executar build, lint e testes disponíveis;
10. mostrar o `git diff`;
11. apresentar o resumo final.

---

## ## RESULTADO ESPERADO

Ao final:

```
```text
/
→ aplicação original funcionando
```
```

```
```text
/lab
→ laboratório SRE funcionando
```
```

E o laboratório deve gerar:

```
```text
health_check
lab_error
lab_slow
```
```

pesquisáveis no Datadog.

## ## RESPOSTA FINAL

Ao concluir, informe objetivamente:

1. arquitetura encontrada;
2. arquivos alterados;
3. alterações realizadas;
4. `service`, `env`, `site` e nome da variável do Client Token;
5. comandos exatos para executar o projeto;
6. URL da aplicação e de `/lab`;
7. consultas para validar os três eventos no Datadog;
8. resultado de build, lint e testes;
9. resumo do `git diff`;
10. limitações restantes.

Comece analisando o projeto.

**\*\*Não altere arquivos antes de entender a arquitetura existente.\*\***

**NÃO É NECESSÁRIO EXECUTAR ESSE PROMPT DURANTE A AULA. O ATUAL PROMPT SERVIRÁ PARA QUE VOCÊ POSSA ADAPTAR O SEU PROJETO FINAL. PARA SEGUIR COM OS EXERCÍCIOS, GASTA CLONAR O REPOSITÓRIO DESCRITO ABAIXO OU FAZER UM GIT PULL NO REPOSITÓRIO EXISTENTE.**

## 1. Preparação do ambiente

Antes de criar monitores ou workflows, valide a aplicação e a chegada dos logs. A maior parte dos erros nesta atividade acontece quando o ambiente ainda não está pronto e o aluno tenta configurar alertas cedo demais.

### 1.1 Clonar e instalar o projeto

1. Clone o repositório pipeline-guardian-agent-demo e entre na pasta do projeto.
2. Confirme que você possui Node.js 22 ou versão compatível com o projeto.
3. Na raiz do repositório, instale as dependências com npm ci.

```
git clone https://github.com/IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo.git
cd pipeline-guardian-agent-demo
npm ci
```

#### POR QUE NPM CI?

O repositório possui package-lock.json. O npm ci reproduz as versões registradas no lockfile e é a opção mais previsível para a aula.

### 1.2 Configurar o Client Token do Datadog

A integração com o Datadog já foi preparada no frontend. Para a aula, você precisa informar apenas o Client Token usado pelo Browser Logs SDK.

4. Na pasta frontend, crie ou edite o arquivo .env.
5. Adicione a variável VITE\_DATADOG\_CLIENT\_TOKEN com o token fornecido pelo professor.
6. Salve o arquivo. Se a aplicação já estiver em execução, reinicie o Vite para que a variável seja recarregada.

```
VITE_DATADOG_CLIENT_TOKEN=<seu-client-token>
```

#### SEGURANÇA

O Client Token do Datadog é o identificador apropriado para coleta no navegador. Não coloque Datadog API Key, Application Key, senha ou outro segredo no frontend.

### 1.3 Executar a aplicação

```
npm run dev
```

A execução em desenvolvimento sobe o backend e o frontend em paralelo. Para este laboratório, abra:

```
http://localhost:5173/lab
```

Na página /lab, confirme que existem três ações:

- Gerar evento saudável
- Simular erro
- Simular latência

## 2. Validar os primeiros logs

Não avance para os cenários enquanto os três tipos de evento não aparecerem no Datadog.



| Ação no /lab          | Evento esperado | Campos importantes                                           |
|-----------------------|-----------------|--------------------------------------------------------------|
| Gerar evento saudável | health_check    | event_type=health_check,<br>route=/lab                       |
| Simular erro          | lab_error       | status_code=500,<br>error_type=SimulatedError,<br>route=/lab |
| Simular latência      | lab_slow        | duration_ms=2500,<br>status_code=200, route=/lab             |

- Abra o Datadog e acesse Logs → Log Explorer.
- Deixe o período de busca incluindo os últimos minutos.
- No /lab, clique uma vez em Gerar evento saudável.
- Pesquise o evento usando a consulta abaixo e abra o resultado.
- Repita o processo para Simular erro e Simular latência.

```
service:copa-figurinhas @event_type:health_check
service:copa-figurinhas @event_type:lab_error
service:copa-figurinhas @event_type:lab_slow
```

### CRITÉRIO DE PRONTIDÃO

Só inicie o Cenário 01 quando você localizar health\_check, lab\_error e lab\_slow no Log Explorer.

## 2.1 Se nenhum log aparecer

| Verificação | O que fazer                                                                                                         |
|-------------|---------------------------------------------------------------------------------------------------------------------|
| Aplicação   | Confirme se <code>http://localhost:5173/lab</code> está aberto e respondendo.                                       |
| .env        | Confirme <code>VITE_DATADOG_CLIENT_TOKEN</code> em <code>frontend/.env</code> e reinicie <code>npm run dev</code> . |
| Consulta    | Teste primeiro apenas <code>service:copa-figurinhas</code> , sem <code>@event_type</code> .                         |
| Período     | Amplie o intervalo do Log Explorer para incluir o momento do clique.                                                |
| Navegador   | Abra DevTools → Console/Network e procure erros relacionados ao envio de logs.                                      |

## 3. Cenário SRE 01: observar antes de alertar

### OBJETIVO

Distinguir um sinal saudável de um sinal de erro antes de criar qualquer regra automática.

- No /lab, clique em Gerar evento saudável três vezes.
- No Log Explorer, pesquise `service:copa-figurinhas @event_type:health_check` e confirme os três eventos.
- Abra um `health_check` e observe `service`, `env`, `event_type` e `route`.
- Volte ao /lab e clique uma vez em Simular erro.

16. Pesquise service:copa-figurinhas @event\_type:lab\_error.
17. Abra o lab\_error e localize status\_code=500, error\_type=SimulatedError e route=/lab.
18. Compare os dois eventos.

| Evidência      | health_check   | lab_error        |
|----------------|----------------|------------------|
| Nível esperado | info           | error            |
| event_type     | health_check   | lab_error        |
| status_code    | não é o foco   | 500              |
| Interpretação  | sinal saudável | falha controlada |

#### PONTO-CHAVE

Um log de erro ainda é apenas um sinal. Não existe ALERT até que um monitor aplique um critério sobre esses eventos.

## 4. Cenário SRE 02: provocar degradação controlada

#### OBJETIVO

Gerar vários erros em uma janela curta para criar um padrão observável, sem depender de falha aleatória.

19. Mantenha o Log Explorer com service:copa-figurinhas @event\_type:lab\_error.
20. No /lab, clique em Simular erro cinco vezes, com aproximadamente um segundo entre os cliques.
21. Aguarde a ingestão e confirme os eventos no período selecionado.
22. Abra dois eventos e confirme que os campos continuam consistentes.
23. Observe a concentração dos eventos no gráfico ou na contagem do Log Explorer.

#### RESULTADO ESPERADO

Você consegue provar que houve degradação observável, mas ainda não existe regra que transforme essa degradação em ALERT.

## 5. Cenário SRE 03: transformar erros em ALERT

#### OBJETIVO

Criar um Log Monitor que transforme três ou mais lab\_error em uma condição operacional explícita.

### 5.1 Criar o Log Monitor de erros

24. No Datadog, abra Monitoring → Monitors → New Monitor e escolha Logs.
25. Use a consulta service:copa-figurinhas @event\_type:lab\_error.
26. Configure a agregação como COUNT.
27. Use uma janela curta. Para reproduzir este laboratório, utilize Past 5 Minutes.
28. Configure Trigger when the evaluated value is above the threshold.
29. Em Alert threshold, informe 2. Assim, 3 ou mais eventos na janela geram ALERT.
30. Nomeie o monitor como [LAB] CopaFigurinhas - erros controlados.

31. Na mensagem, use algo simples como: Falhas controladas detectadas no serviço CopaFigurinhas.
32. Salve o monitor.

| Configuração    | Valor                                         |
|-----------------|-----------------------------------------------|
| Query           | service:copa-figurinhas @event_type:lab_error |
| Agregação       | COUNT                                         |
| Janela          | Past 5 Minutes                                |
| Alert threshold | > 2                                           |
| Nome            | [LAB] CopaFigurinhas - erros controlados      |

## 5.2 Provocar OK → ALERT

33. Espere o monitor ficar em OK.
34. Volte ao /lab e clique em Simular erro pelo menos três vezes dentro da janela.
35. Confirme os lab\_error no Log Explorer.
36. Volte ao monitor e aguarde a avaliação.
37. Confirme a mudança de OK para ALERT.

### LEITURA DO CENÁRIO

LOG é o sinal. MONITOR é o critério. ALERT é o resultado do critério.

# 6. Cenário SRE 04: ALERT dispara resposta low-code

### OBJETIVO

Executar a cadeia completa: lab\_error → Log Monitor → ALERT → Monitor Trigger → If condition → JavaScript seguro.

## 6.1 Criar o Workflow Automation

38. Abra Automation → Workflow Automation.
39. Clique em New Workflow e inicie do zero.
40. Use um nome fácil de reconhecer, por exemplo CopaFigurinhas - resposta a erros controlados.
41. Adicione um Monitor Trigger.

## 6.2 Adicionar a decisão

Na versão atual da interface, a ação de decisão que usamos é If condition.

42. Clique no + abaixo do Monitor Trigger.
43. Pesquise If condition e adicione a ação.
44. Abra a aba Context Variables da condição.
45. Expanda Source → monitor e localize overall\_state.
46. Volte para Configure e monte a comparação com o estado Alert.

```
{{ Source.monitor.overall_state }} = Alert
```

### POR QUE USAR CONTEXT VARIABLES?

Os dados disponíveis podem variar conforme o trigger. Escolher a variável pela própria interface reduz erros de caminho e deixa claro qual dado está sendo avaliado.

## 6.3 Adicionar a ação segura

47. No caminho TRUE do If condition, clique em +.
48. Adicione JavaScript em Data Transformation.
49. Use uma resposta simples, sem reiniciar serviços, fazer deploy, excluir recursos ou alterar banco.

```
return {
 action: "investigate",
 service: "copa-figurinhas",
 reason: "controlled_error_threshold_exceeded"
};
```

O caminho FALSE pode permanecer sem ação neste laboratório.

## 6.4 Publicar o workflow

50. Clique em Publish.
51. Para este laboratório, mantenha Run as = Owner.
52. Em Who has access, Only me é suficiente para uma execução individual.
53. Clique em Publish.

## 6.5 Associar o monitor ao workflow

54. Abra Monitoring → Monitors e selecione [LAB] CopaFigurinhas - erros controlados.
55. Clique em Edit.
56. Vá até Configure notifications & automations.
57. Na área Attached Automation, clique em Add Workflow.
58. Selecione o workflow recém-publicado.
59. Confirme que Attached Automation mostra 1 workflow associado.
60. Clique em Save and Publish no monitor.

### IMPORTANTE

O workflow precisa estar publicado e associado ao monitor para que o ALERT real dispare a automação.

## 6.6 Validar ponta a ponta

61. Espere o monitor voltar para OK.
62. No /lab, gere pelo menos três lab\_error dentro da janela.
63. Confirme que o monitor entra em ALERT.
64. Abra Automation → Workflow Automation → seu workflow → Executions.
65. Abra a execução mais recente disparada pelo monitor.
66. Confirme as três etapas com sucesso: Monitor, If condition e JavaScript.
67. Abra o output do JavaScript.

```
{
 "action": "investigate",
 "reason": "controlled_error_threshold_exceeded",
 "service": "copa-figurinhas"
}
```

**COMO INTERPRETAR**

O Monitor detectou que o critério foi ultrapassado. O Workflow recebeu o estado Alert, o If condition avaliou TRUE e o JavaScript produziu uma resposta estruturada. A automação não corrigiu a aplicação; ela executou uma decisão segura e rastreável.

**SOBRE TEST FROM MONITOR**

Use Test from Monitor como apoio de depuração quando necessário. Para considerar o cenário concluído, priorize a validação real OK → ALERT → execução do workflow, pois ela comprova a integração completa.

## 7. Cenário SRE 05: latência controlada

**OBJETIVO**

Mostrar que uma requisição pode responder com status 200 e ainda assim apresentar degradação de desempenho.

### 7.1 Observar o sinal de latência

68. No /lab, clique em Simular latência.
69. Aguarde aproximadamente 2,5 segundos.
70. No Log Explorer, pesquise service:copa-figurinhas @event\_type:lab\_slow.
71. Abra o evento e localize duration\_ms=2500, status\_code=200, event\_type=lab\_slow e route=/lab.
72. Repita duas ou três vezes.

**INTERPRETAÇÃO**

status\_code=200 indica que houve resposta, mas duration\_ms=2500 mostra que a experiência está degradada. Erro e latência precisam de critérios diferentes.

### 7.2 Criar um Log Monitor de latência

Para a aula, monitore apenas eventos lab\_slow cuja duração seja superior a 2000 ms e gere ALERT quando houver três ou mais ocorrências em 5 minutos.

```
service:copa-figurinhas @event_type:lab_slow @duration_ms:>2000
```

| Configuração    | Valor                                                           |
|-----------------|-----------------------------------------------------------------|
| Query           | service:copa-figurinhas @event_type:lab_slow @duration_ms:>2000 |
| Agregação       | COUNT                                                           |
| Janela          | Past 5 Minutes                                                  |
| Alert threshold | > 2                                                             |
| Nome            | [LAB] CopaFigurinhas - latência controlada                      |

**ATENÇÃO AO THRESHOLD**

O valor 2000 ms fica na query. Como o monitor está agregando COUNT, o Alert threshold deve ser 2, e não 2000. Assim, três eventos lentos geram ALERT.

**7.3 Criar o workflow de latência**

Para manter a explicação simples, crie um segundo workflow seguindo o mesmo padrão do cenário anterior.

73. Crie um workflow com Monitor Trigger.
74. Adicione If condition com `{{ Source.monitor.overall_state }}` = Alert.
75. No TRUE, adicione JavaScript.
76. Publique o workflow e associe-o ao monitor [LAB] CopaFigurinhas - latência controlada.

```
return {
 action: "investigate",
 service: "copa-figurinhas",
 reason: "latency_threshold_exceeded"
};
```

**7.4 Validar o cenário**

77. Espere o monitor ficar em OK.
78. Clique em Simular latência pelo menos três vezes dentro da janela.
79. Confirme os lab\_slow no Log Explorer.
80. Confirme o monitor em ALERT.
81. Abra a execução do workflow e verifique Monitor → If condition TRUE → JavaScript.
82. Confirme reason=latency\_threshold\_exceeded no output.

**8. Modelo mental do laboratório**

Use esta sequência para entender qualquer um dos cenários:

| Etapas       | Pergunta que você deve responder                    |
|--------------|-----------------------------------------------------|
| 1. Sinal     | Qual evento foi produzido pela aplicação?           |
| 2. Evidência | Quais campos do log comprovam o que ocorreu?        |
| 3. Critério  | Qual query, janela e threshold o monitor avalia?    |
| 4. Alerta    | O monitor realmente mudou para ALERT?               |
| 5. Disparo   | Qual monitor iniciou o workflow?                    |
| 6. Decisão   | Qual condição foi avaliada e qual caminho executou? |
| 7. Ação      | Qual output ou ação segura foi produzida?           |

**8.1 Troubleshooting rápido**

| Problema                                 | Verificação rápida                                                                            |
|------------------------------------------|-----------------------------------------------------------------------------------------------|
| Nenhum log chega                         | Revise frontend/.env, reinicie npm run dev, amplie o período e teste service:copa-figurinhas. |
| health_check chega, mas outro evento não | Clique novamente no botão correspondente e abra DevTools para verificar erro no navegador.    |

|                                          |                                                                                                                    |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Monitor não alerta                       | Confirme query no Log Explorer, janela, COUNT e threshold > 2. Gere 3 ou mais eventos dentro da janela.            |
| Workflow não dispara                     | Confirme Published, Monitor Trigger e Attached Automation no monitor.                                              |
| If condition executa, mas JavaScript não | Abra a execução real do monitor e confira Source.monitor.overall_state. O JavaScript está somente no caminho TRUE. |
| JavaScript não retorna objeto            | Use return { ... }; no script.                                                                                     |
| Monitor demora para voltar a OK          | A janela ainda contém eventos que ultrapassam o threshold. Aguarde a janela expirar.                               |

## 9. Atividade dos alunos: desafio com 3 cenários

### PROPOSTA

Agora você vai reutilizar o que aprendeu, mas deverá modificar a lógica do Workflow Automation. Não basta repetir exatamente os workflows do laboratório.

Trabalhe apenas com ações seguras. Não reinicie serviços, não execute deploy e não altere dados da aplicação.

### Desafio 1: um workflow para dois tipos de alerta

Crie um novo workflow que possa ser associado aos dois monitores já existentes: erros controlados e latência controlada.

- Use o contexto do monitor para identificar qual monitor disparou a execução.
- Crie caminhos diferentes para erro e latência.
- O output deve deixar claro qual tipo de problema foi detectado.

### CRITÉRIO DE SUCESSO

Uma execução disparada pelo monitor de erros deve produzir um output diferente de uma execução disparada pelo monitor de latência.

### Desafio 2: tratar recuperação sem executar investigação

Evolua um workflow para diferenciar estado de alerta e estado que não representa um alerta ativo.

- Quando o estado for Alert, mantenha a resposta de investigação.
- No caminho alternativo, produza uma resposta segura indicando no\_action ou recovered.
- Use Test from Monitor, quando disponível, para experimentar estados diferentes. Depois valide pelo histórico de execução.

### CRITÉRIO DE SUCESSO

O aluno deve demonstrar os dois caminhos e explicar por que a mesma automação não deve investigar quando o monitor não está em Alert.

## Desafio 3: enriquecer a decisão operacional

Crie um workflow que retorne uma resposta mais informativa do que action + reason.

- Inclua uma classificação simples, por exemplo priority=high para erro e priority=medium para latência.
- Mantenha service=copa-figurinhas.
- Inclua um reason coerente com o monitor que disparou o workflow.
- O output deve continuar sendo apenas uma transformação segura de dados.

### CRITÉRIO DE SUCESSO

O output final precisa permitir entender o problema, a prioridade e a ação recomendada sem abrir o código da aplicação.

## 9.1 Evidências mínimas para entrega

| Evidência    | O que mostrar                                          |
|--------------|--------------------------------------------------------|
| Log Explorer | Consulta e evento usado no teste.                      |
| Monitor      | Nome, query, janela, threshold e estado ALERT.         |
| Workflow     | Canvas com trigger, condição e ações.                  |
| Execution    | Etapas executadas e caminho selecionado.               |
| Output       | Objeto final produzido pela ação segura.               |
| Explicação   | Em poucas linhas: input → condição → caminho → output. |

## 10. Checklist final

- ☐ Projeto instalado e npm run dev funcionando.
- ☐ frontend/.env configurado com VITE\_DATADOG\_CLIENT\_TOKEN.
- ☐ http://localhost:5173/lab aberto.
- ☐ health\_check localizado no Log Explorer.
- ☐ lab\_error localizado com status\_code=500.
- ☐ lab\_slow localizado com duration\_ms=2500.
- ☐ Monitor de erros realiza OK → ALERT.
- ☐ Workflow de erros executa Monitor → If condition TRUE → JavaScript.
- ☐ Output de erros contém controlled\_error\_threshold\_exceeded.
- ☐ Monitor de latência usa @duration\_ms:>2000 e threshold COUNT > 2.
- ☐ Workflow de latência retorna latency\_threshold\_exceeded.
- ☐ Você consegue explicar sinal → critério → alerta → decisão → ação.

### REGRA DE OURO

Antes de criar automação, prove o sinal no Log Explorer. Antes de culpar o workflow, prove que o monitor chegou a ALERT. Depois, abra a execução e leia cada etapa, input e output.



